

TOPIC 01: LIBRARY VS FRAMEWORK & WHAT IS EXPRESS?

Concept	Definition & Kaam	Example
Library	Pre-written code ka aisa collection jise hum kisi specific task ko karne ke liye apni marzi se call karte hain. Control humare haath me hota hai.	axios, lodash
Framework	Pre-written code ka pura structure/architecture jiske rules ke mutabiq hume code likhna padta hai (Inversion of Control).	Express.js

Express.js Kya Hai?

Node.js ka ek fast, minimalist web framework jo server-side web applications aur APIs banane ke liye use hota hai.

Express Ke 4 Core Kaam:

1. LISTEN INCOMING REQUESTS



2. PARSE DATA



4. SEND RESPONSE



3. MATCH ROUTES

TOPIC 02: BASIC SERVER SETUP & APP.USE()

```
const express = require("express"); const app = express(); let port = 3000; // Har aane wali request ko listen karta hai (Middleware) app.use((req, res) => { console.log("Request received!"); }); app.listen(port, () => { console.log(`App is listening on port ${port}`); });
```

TOPIC 03: ROUTING, PARAMS & QUERY STRINGS

- **Routing:** Network me aane wali traffic/requests ke URL path aur method ke mutabiq sahi response execute karne ka process.
- **Nodemon:** Server code modify hone par automatically server restart karne wala development tool.
- **Wildcard Route (*):** Agar request kisi bhi path se match na kare toh 404 handler: `app.get("*", ...)`.

```
// 1. Path Parameters (req.params): /ig/:username/:id app.get("/:username", (req, res) => { let { username } = req.params; // URL variable nikalna }); // 2. Query String (req.query): /search?q=apple app.get("/search", (req, res) => { let { q } = req.query; // Query param nikalna });
```

TOPIC 04: EJS (EMBEDDED JAVASCRIPT TEMPLATES)

What is EJS?

EJS ek simple templating language hai jo plain JavaScript ke sath **Dynamic HTML** markup generate karne ki suvidha deti hai.

```
app.set("view engine", "ejs"); // EJS view engine set kiya app.get("/", (req, res) => { res.render("home.ejs"); // views/home.ejs render karega });
```



Views Path Rule: Agar server kisi doosre directory se run kiya jaye toh views error na de, isliye path fix karna zaroori hai:

```
const path = require("path"); app.set("views", path.join(__dirname, "/views"));
```

TOPIC 05: INTERPOLATION & EJS TAGS

Interpolation: HTML markup text ke andar expressions ya variables ki value inject/embed karna.

EJS Tag	Meaning & Behavior	Example
<%= %>	Outputs value: Variable ya calculation evaluate karke print karta hai (HTML escaped).	<%= name %>
<% %>	Scriptlet: JS code flow (if-else, loops) chalata hai, screen pe print nahi karta.	<% if(age >= 18) { %>
<%- %>	Unescaped HTML: Raw HTML render karta hai aur partial sub-templates include karta hai.	<%- include(...) %>
<%# %>	Comment: Client browser ko send nahi hota, sirf server file me comment rehta hai.	<%# This is comment %>

TOPIC 06: PASSING DATA & INCLUDES (PARTIALS)

```
// 1. Server se EJS ko data bhejna app.get("/rolldice", (req, res) => { let diceVal = Math.floor(Math.random() * 6) + 1; res.render("rolldice.ejs", { num: diceVal }); });
// 2. EJS me Sub-templates (Includes) use karna: // views/home.ejs ke andar navbar/header joDNA: <%- include("includes/head.ejs") %>
```


TOPIC 07: GET VS POST REQUEST

Feature	GET Request	POST Request
Purpose	Server se data retrieve/fetch karna	Server par data create/write/update karna
Data Location	URL Query String me jata hai (?key=val)	HTTP Request Body (req.body) me jata hai
Security	Sensitive data ke liye unsafe (URL me dikhta hai)	Secure: URL me data nahi dikhta
Data Limit	Limited (Length restrictions by browsers)	Unlimited / Large files send kar sakte hain

TOPIC 08: PARSING POST REQUEST BODY

Why Body Parser is Needed?

Express by default request body ke data ko parse nahi karta aur `req.body` undefined hota hai. Isliye middleware lagana zaroori hai:

```
// 1. HTML Form Data (URL-encoded) ko parse karne ke liye:
app.use(express.urlencoded({ extended: true })); // 2. JSON Data (API requests) ko
parse karne ke liye: app.use(express.json()); // POST Route handler:
app.post("/register", (req, res) => { let { username, password } = req.body;
res.send(`Welcome ${username}!`); });
```



Key Takeaway: `extended: true` lena se nested objects bhi safely parse ho jaate hain!

